

Dynamic Programming for Shortest Paths in Graphs

Did somebody
say G-rap h?



Have a meaningful Yom Kippur!

- Other Holidays:
 - Earth, Wind, & Fire Day
 - Escapology Day
 - Pause the World Day
 - National Sponge Candy Day
 - World Gratitude Day
 - National Beef Stroganoff Day
 - Miniature Golf Day
 - National Chai Day
 - National Pecan Cookie Day
 - International Day of Peace
 - World Alzheimer's Day
 - National Farm Safety Day for Kids
- This Week is Sea Otter Awareness Week
- September is National Sickle Cell Awareness Month

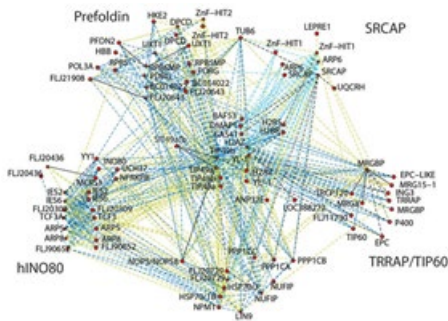
Why Graphs?



The Internet



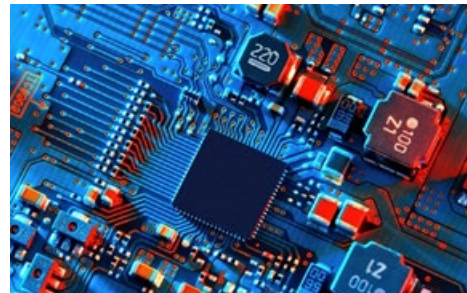
Social Networks



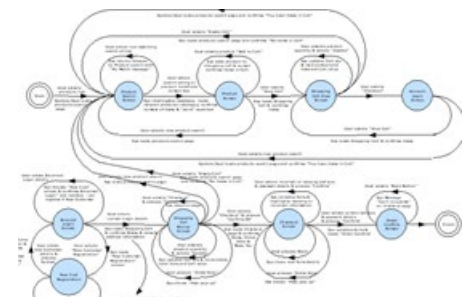
Biological and Chemical Networks



Transportation Networks



Circuits



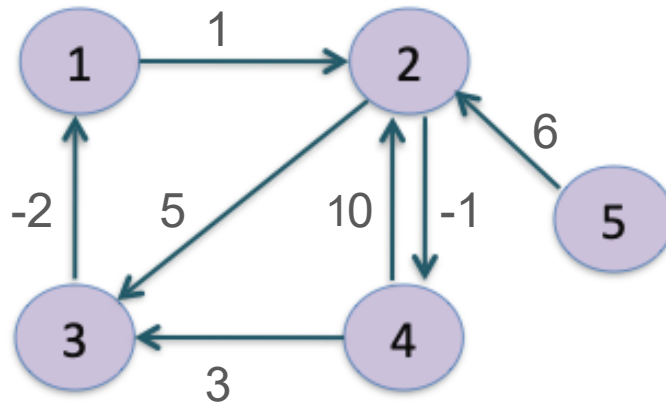
State Transition Networks

Directed and undirected graphs

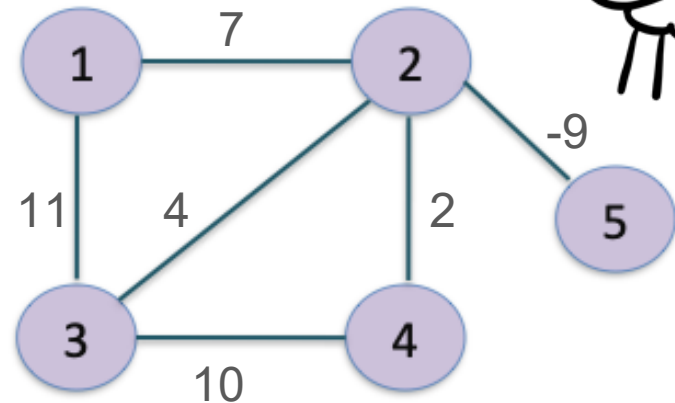
Why do we even care about negative weights?



Directed graph



Undirected graph



$\text{dist}(s, t)$ = minimum, over all paths P from s to t , of the sum of the edge weights along P .

$\text{dist}(s, t) = \infty$ if there is no path from s to t .

$\text{dist}(s, t)$ = undefined (alternatively, $-\infty$) if there is no shortest path

Representation: Adjacency lists

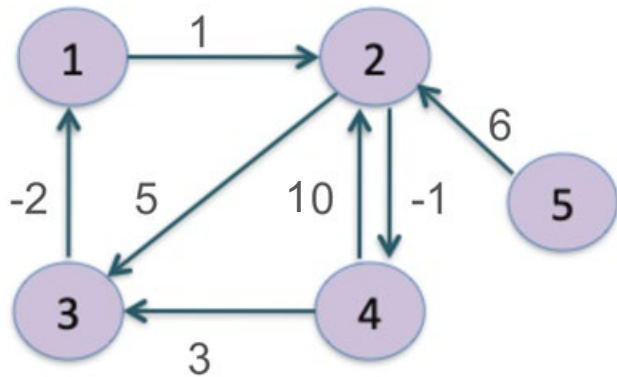
Notation: V = vertex set, E = edge set, $n = |V|$, $m = |E|$

The shortest-path problems we'll consider

Input: Weighted directed graph. Weights can be negative, but *assume no negative-weight cycles*.

Single-Source Shortest Paths (SSSP): Given a “source” vertex s , find a shortest path from s to every vertex t .

All-Pairs Shortest Paths (APSP): For every pair s, t of vertices, find a shortest path from s to t .



	1	2	3	4	5
1	0	1	3	0	∞
2	0	0	2	-1	∞
3	-2	-1	0	-2	∞
4	1	2	3	0	∞
5	6	6	8	5	0

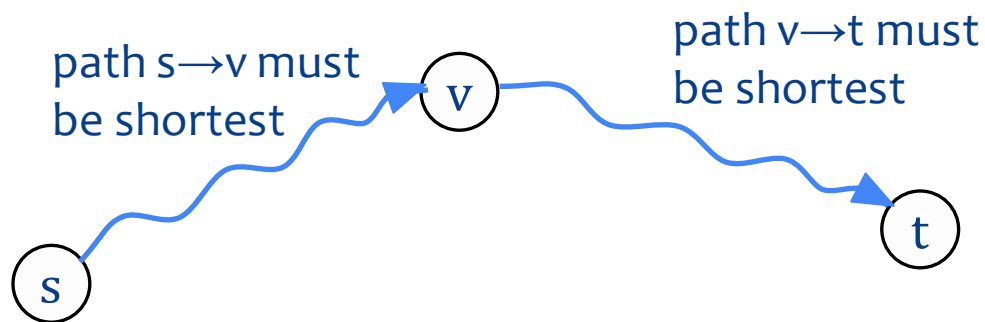
What about
single-pair
shortest path?



First Key Observation

Principle of
Optimality

1. If a shortest path from s to t goes through vertex v , then it must be a **shortest path from s to v** , then a **shortest path from v to t** .



The DP Recipe

1. Define *value version* of what we want to compute
2. Derive a recurrence
3. Size of table: How many dimensions? Range of each dimension?
4. What are the base case(s)?
5. To fill in a cell, which other cells need to be filled already? In which order do I fill the table?
6. Which cell(s) contain the final answer?
7. Running time is often (size of table) $\cdot$ (time to fill each entry)
8. To reconstruct a solution (instead of just its value) follow arrows from final answer to base case

Check out my recurrence for SSSP!

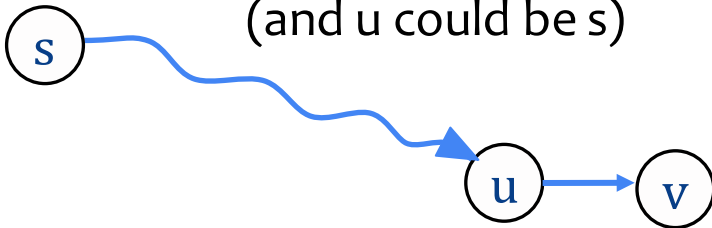


Your equation is technically true, but it's not a legal recurrence (no clear order), so it's not suitable for DP.

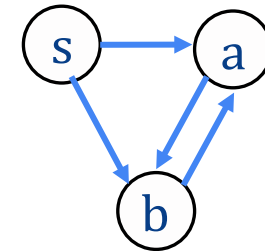
$$\text{dist}(s, v) = \min_{(u, v) \in E} \{ \text{dist}(s, u) + w(u, v) \}$$

What order should we compute these?

In a shortest $s \rightarrow v$ path, u is the last vertex before v (and u could be s)



Base case: $\text{dist}(s, s) = 0$



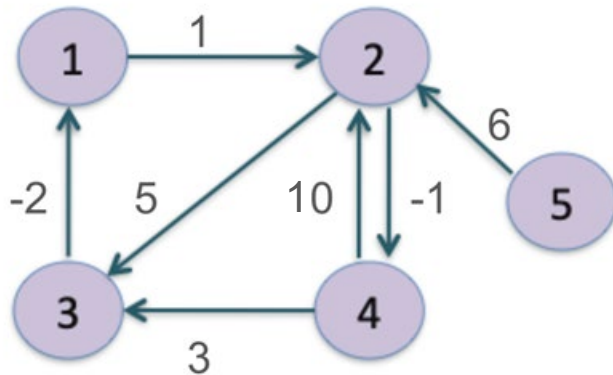
$\text{dist}(s, a)$ depends on $\text{dist}(s, b)$
 $\text{dist}(s, b)$ depends on $\text{dist}(s, a)$

Notation:

- $w(y, z)$ is the weight (or “cost”) of edge (y, z) .
- $\text{dist}(y, z)$ is the distance from y to z .

Recurrence for SSSP

Key Definition. Let $\text{dist}^{(i)}(s, v)$ be the *length* of a shortest s-to-v path that uses at most i edges, or ∞ if there's no such path.



$$\text{dist}^{(0)}(5, 3)? = \infty$$

$$\text{dist}^{(1)}(5, 3)? = \infty$$

$$\text{dist}^{(2)}(5, 3)? = 11$$

$$\text{dist}^{(3)}(5, 3)? = 8$$

$$\text{dist}^{(4)}(5, 3)? = 8$$

$$\text{dist}^{(5)}(5, 3)? = 8$$

$$\text{dist}^{(6)}(5, 3)? = 8$$

$$\text{dist}^{(7)}(5, 3)? = 8$$

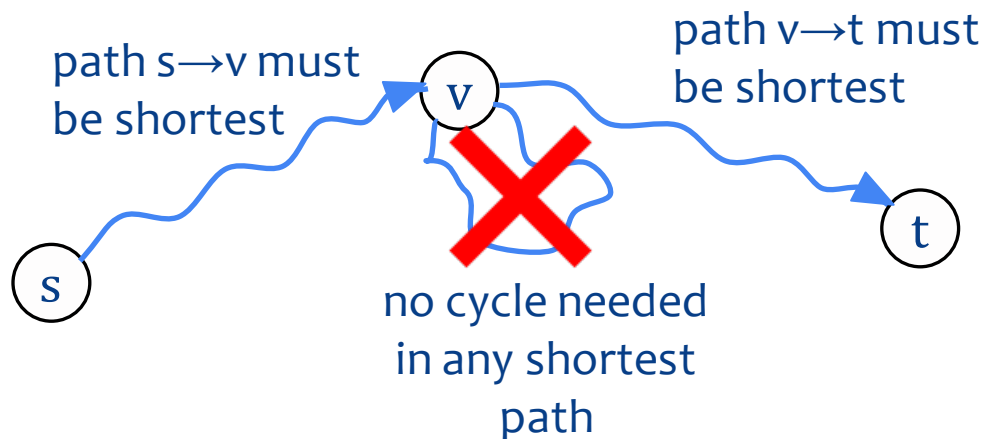
When can we stop?

Why is it enough to only compute up to $\text{dist}^{(4)}(s, v)$?

Second Key Observation

Principle of
Optimality

1. If a shortest path from s to t goes through vertex v , then it must be a **shortest path from s to v** , then a **shortest path from v to t** .
2. Since there is no negative-weight cycle in the graph, there is a shortest path from s to t with **no cycle** in it.



Recurrence for SSSP

Key Definition. Let $\text{dist}^{(i)}(s, v)$ be the *length* of a shortest s-to-v path that uses at most i edges, or ∞ if there's no such path.

Ultimate goal: $\text{dist}^{(n-1)}(s, v)$ for each v . Why?

Recurrence:

$$\text{dist}^{(i)}(s, v) = \min \left\{ \text{dist}^{(i-1)}(s, v), \min_{(u,v) \in E} \{ \text{dist}^{(i-1)}(s, u) + w(u, v) \} \right\}$$

Base case(s):

$$\text{dist}^{(0)}(s, s) = 0$$

$$\text{For all } v \neq s, \text{dist}^{(0)}(s, v) = \infty$$

The recurrence takes
inspiration from Professor
Y's first attempt



This yields the **Bellman-Ford Algorithm**
(Bellman, Ford, Moore, Shimbel, 1955)

Let's follow the DP Recipe

This is called the **Bellman-Ford Algorithm**
(Bellman, Ford, Moore, Shimbel, 1955)

Pseudocode for Bellman-Ford

Algorithm SSSP(G, s)

table := 2D-array indexed from 0 to $n-1$ in both dimensions

// first dimension represents vertices $v_0, \dots, v_{n-1}$ **where $s=v_0$** ,
second dimension represents the number i of edges in paths

table(0,0) = 0 // base case for s

for $v = 1$ to $n-1$: **table**($v, 0$) = ∞ // rest of base cases

for $i = 1$ to $n-1$: // for each number i of edges

for $v = 0$ to $n-1$: // for each vertex

table(v, i) = $\min\{\mathbf{table}(v, i-1), \min_{(u,v) \in E} \{\mathbf{table}(u, i-1) + w(u,v)\}\}$

Return, for each $v \in V$, **table**($v, n-1$) as $\text{dist}(s, v)$

Claim: Running time is $\Theta(|V| \cdot |E|) = \Theta(nm)$. Why?

This algorithm is designed under the assumption that the graph contains no negative-weight cycle. **But maybe it does?**

With a slight modification, the Bellman-Ford (BF) Algorithm can **detect** whether there is a negative-weight cycle:

Run BF for 1 more iteration!

I.e., calculate $\text{dist}^{(i)}(s, v)$ for all i up to n .

Claim: There is a negative-weight cycle (reachable from s) if and only if $\text{dist}^{(n)}(s, v) < \text{dist}^{(n-1)}(s, v)$, for some v .

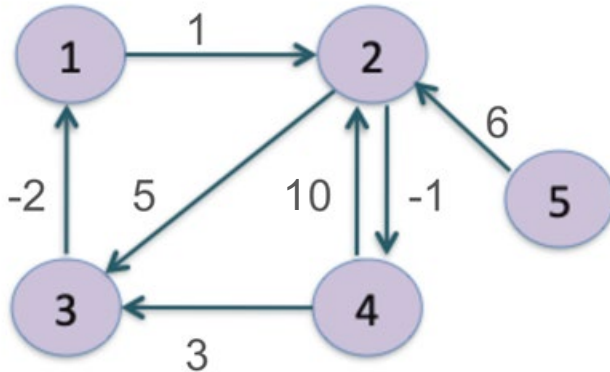
Proof idea: recurrence “discovers” a neg-wt cycle, which reduces obtainable distance to some vertex on that cycle.

Faster Algorithms for SSSP

- Bernstein, Nanongkai, Wulff-Nilsen ('22): $O(m \cdot \log^8 n)$ ← integer weights
- Fineman ('24), ..., Hair, Li, Li, Zhang ('26): $O(m^{1+o(1)})$ ← any weights
- If no negative weights:
 - Dijkstra's algorithm ('59): $O(m + n \log n)$
 - Duan, Mao, Mao, Shu, Yin ('26): $O(m \cdot \log^{1/2} n + \sqrt{mn} \cdot \log n \cdot \log \log n)$

All-Pairs Shortest Paths (APSP)

All-Pairs Shortest Paths (APSP): For every pair s, t of vertices, find a shortest path from s to t .



	1	2	3	4	5
1	0	1	3	0	∞
2	0	0	2	-1	∞
3	-2	-1	0	-2	∞
4	1	2	3	0	∞
5	6	6	8	5	0

Initial idea for solving APSP: Run SSSP from every vertex!

That works, but one algorithm you'll see soon is faster for *dense* graphs: $O(n^3)$ instead of $\Theta(mn^2)$ (better when $m \gg n$).

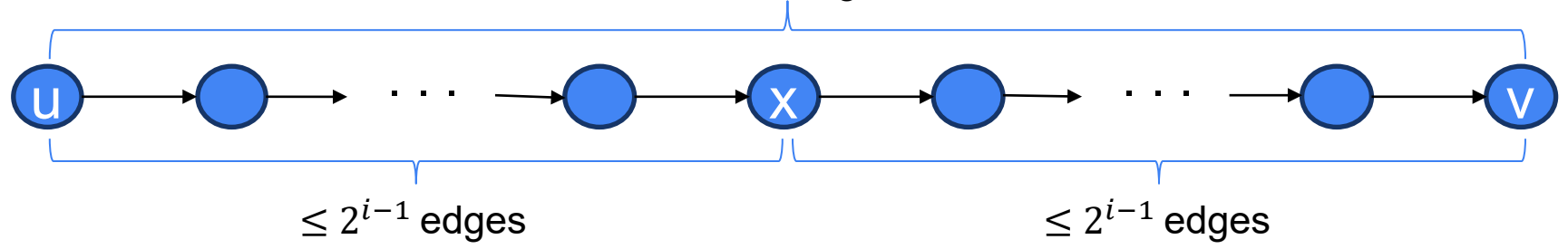
Can we combine dynamic programming & divide and conquer?

YES!

- Bellman-Ford recurrence: $dist^{(i)}(s, v) = \min_{(u,v) \in E} \{dist^{(i-1)}(s, u) + w(u, v)\}$



- Why reduce the path length by just $\leq 2^i$ edges?



- “Path doubling” recurrence:

$$dist^{(2^i)}(u, v) = \min_{x \in V} \{dist^{(2^{i-1})}(u, x) + dist^{(2^{i-1})}(x, v)\}$$

- Base case(s): $dist^{(2^0)}(u, v) = \begin{cases} 0 & \text{if } u = v \\ w(u, v) & \text{if } (u, v) \in E \\ \infty & \text{if } (u, v) \notin E \end{cases}$

Pseudocode for Path-Doubling APSP

Algorithm PathDoublingAPSP(G, s) // vertices indexed $v_0, \dots, v_{n-1}$
 table := 3D-array with dimensions $n \times n \times \lceil \log(n-1) \rceil$.
 for $u = 0$ to $n-1$:
 for $v = 0$ to $n-1$:
 if ($u=v$) **table**($u, u, 0$) = 0 // base case for $dist^{(0)}(u, u)$
 if $(u, v) \in E$, **table**($u, v, 0$) = $w(u, v)$ // base case
 else **table**($u, v, 0$) = ∞ // base case
 for $i = 1$ to $\log(n-1)$: // for each number i of edges
 for $u = 0$ to $n-1$:
 for $v = 0$ to $n-1$:
 table(u, v, i) = $\min_{x \in V} \{ \mathbf{table}(u, x, i-1) + \mathbf{table}(x, v, i-1) \}$
 Return **table**($u, v, \log(n-1)$) as $dist(u, v)$.

What is the running time? ((table cells) $\times$ (time to fill cell)): $\Theta(n^3 \log n)$

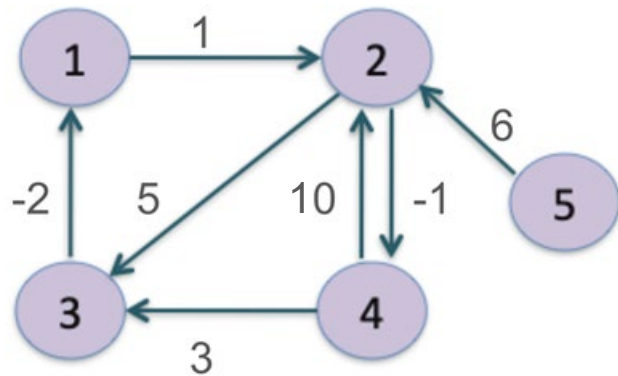
Old idea: “ i ” parameter counts **length** of permissible paths.

New idea: Order vertices arbitrarily as $v_1, v_2, \dots, v_n$. New “ i ” parameter says **allowable intermediate vertices** are $v_1, \dots, v_i$.

Key Definition:

$\text{dist}^{[i]}(s, t)$ = length of shortest s - t path that is allowed to use vertices $\{s, t\} \cup \{v_1, \dots, v_i\}$ (not necessarily all of them).

What is...



$$\text{dist}^{[0]}(5, 3)? = \infty$$

$$\text{dist}^{[1]}(5, 3)? = \infty$$

$$\text{dist}^{[2]}(5, 3)? = 11$$

$$\text{dist}^{[3]}(5, 3)? = 11$$

$$\text{dist}^{[4]}(5, 3)? = 8$$

$$\text{dist}^{[5]}(5, 3)? = 8$$

Old idea: “ i ” parameter counts length of permissible paths.

New idea: Order vertices arbitrarily as $v_1, v_2, \dots, v_n$. New “ i ” parameter says *allowable intermediate vertices* are $v_1, \dots, v_i$.

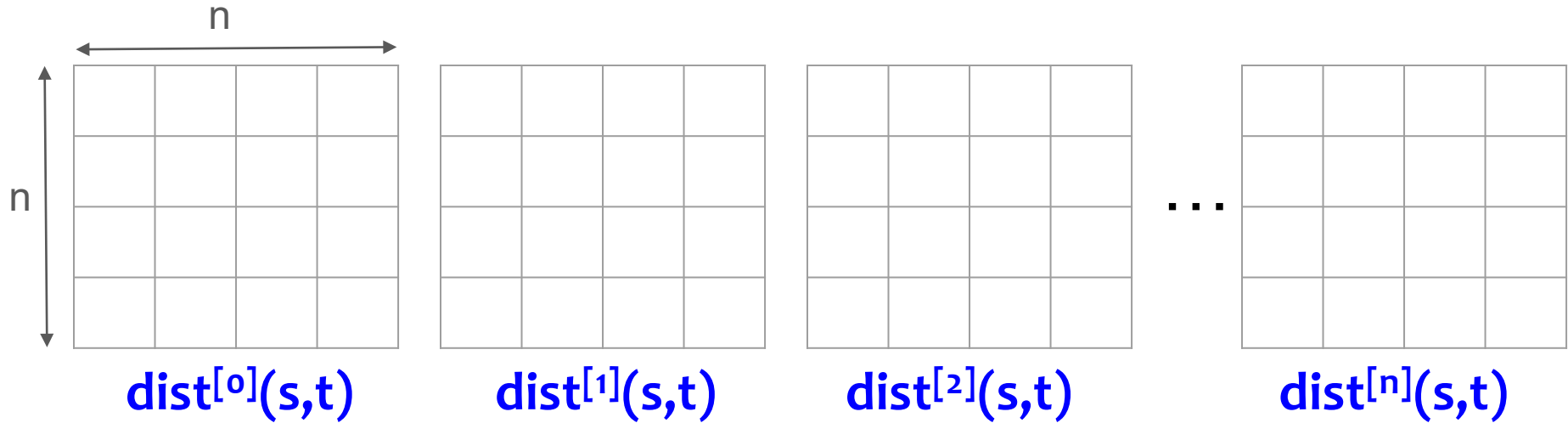
Key Definition:

$dist^{[i]}(s, t)$ = length of shortest s - t path that is allowed to use vertices $\{s, t\} \cup \{v_1, \dots, v_i\}$ (not necessarily all of them).

$$dist^{[i]}(s, t) = \min \left\{ \underset{\text{Use it!}}{dist^{[i-1]}(s, i) + dist^{[i-1]}(i, t)}, \quad \underset{\text{Or lose it!}}{dist^{[i-1]}(s, t)} \right\}$$

$$\text{Base case: } dist^{[0]}(s, t) = \begin{cases} 0 & \text{if } s = t \\ w(s, t) & \text{if } (s, t) \in E \\ \infty & \text{otherwise} \end{cases}$$

Let's Follow the DP Recipe



This is called the **Floyd-Warshall Algorithm** (1962)

Pseudocode for Floyd–Warshall

Algorithm APSP(G)

table := 3D-array ($1..n, 1..n, 0..n$)

// first two dimensions represent vertices $v_1, \dots, v_n$,

third dimension represents restricting to the first i internal vertices

for $s = 1$ to n :

for $t = 1$ to n :

table($s, t, 0$) = 0 or $w(s, t)$ or ∞ when $s = t$, $(s, t) \in E$, and $(s, t) \notin E$.

for $i = 1$ to n :

for $s = 1$ to n :

for $t = 1$ to n :

table(s, t, i) = $\min\{\mathbf{table}(s, t, i-1), \mathbf{table}(s, i, i-1) + \mathbf{table}(i, t, i-1)\}$

Return **table**(s, t, n) for all s, t

Running time = ((array cells) $\times$ (time to fill one cell)) = $\Theta(n^3)$

A question to ponder...

Why doesn't the trick of computing $\text{dist}^{[i]}(s,t)$ give a faster-than-Bellman-Ford algorithm for SSSP too?

Progress on APSP since Floyd-Warshall

Author	Runtime	Year
Fredman	$n^3 \log \log^{1/3} n / \log^{1/3} n$	1976
Takaoka	$n^3 \log \log^{1/2} n / \log^{1/2} n$	1992
Dobosiewicz	$n^3 / \log^{1/2} n$	1992
Han	$n^3 \log \log^{5/7} n / \log^{5/7} n$	2004
Takaoka	$n^3 \log \log^2 n / \log n$	2004
Zwick	$n^3 \log \log^{1/2} n / \log n$	2004
Chan	$n^3 / \log n$	2005
Han	$n^3 \log \log^{5/4} n / \log^{5/4} n$	2006
Chan	$n^3 \log \log^3 n / \log^2 n$	2007
Han, Takaoka	$n^3 \log \log n / \log^2 n$	2012
Williams	$n^3 / \exp(\sqrt{\log n})$	2014



Get a load
of all those
logs!!



Conclusion: Maybe $O(n^{2.999})$ is impossible?

Maybe $O(n^{2.999})$ is impossible?

Either **ALL** of the following have $O(n^{<3})$ time algorithms or **NONE** of them do: (Virginia Vassilevska Williams, Ryan Williams, 2010)

1. APSP
2. Minimum Weight Triangle
3. Metricity
4. Minimum Cycle
5. Distance Product
6. Second Shortest Path
7. Replacement Paths
8. Negative Triangle Listing

...



Appendix: Optimizing Bellman-Ford and Floyd-Warshall

Appendix: Optimizing Bellman-Ford and Floyd-Warshall

- In practice you **don't** need to store an $n \times n$ table to run Bellman-Ford, nor do you need an $n \times n \times n$ table to implement Floyd-Warshall.

Algorithm SSSP(G, s)

table := 1D-array indexed from 0 to $n-1$.

// vertices indexed as $v_0, \dots, v_{n-1}$ **where $s = v_0$** ,

table(0) = 0 // base case for s

for $v = 1$ to $n-1$: **table**(v) = ∞ // rest of base cases

repeat $n - 1$ times (*)

 for each edge $(u, v) \in E$

table(v) = $\min\{\text{table}(v), \text{table}(u) + w(u, v)\}$

Return, for each $v \in V$, **table**(v) as $\text{dist}(s, v)$

- Claim:** After i iters. of (*), $\text{dist}(s, v) \leq \text{table}(v) \leq \text{dist}^{(i)}(s, v)$.

Appendix: Optimizing Bellman-Ford and Floyd-Warshall

- In practice you **don't** need to store an $n \times n$ table to run Bellman–Ford, nor do you need an $n \times n \times n$ table to implement Floyd–Warshall.

Algorithm APSP(G)

table := 2D-array (1.. n , 1.. n)

for $s = 1$ to n , for $t = 1$ to n : // base cases

$\text{table}(s, t, 0) = 0$ or $w(s,t)$ or ∞ when $s = t$, $(s, t) \in E$, and $(s, t) \notin E$.

for $i = 1$ to n , for $s = 1$ to n , for $t = 1$ to n :

$\text{table}(s,t) = \min\{\text{table}(s,t), \text{table}(s,i) + \text{table}(i,t)\}$

 //note: third arg. omitted

Return $\text{table}(s, t)$ for all s, t

- Claim:** after each iteration of the outer i -loop,
 $\text{table}(s,t) = \text{dist}^{[i]}(s, t)$.